COLLEGE PROJECT DOCUMENTATION

RAG Assistant: A Retrieval-Augmented Generation Document Assistant

Project Plan

Document	Project Plan
Version	1.0
Date prepared	12 October 2026
Prepared by	
Course / Supervisor	

Planning document. The project is estimated from Day 0: no features are assumed to be built, and every activity in this document is planned work, not completed work.

Contents

1. Project Introduction and Objectives
2. Scope
3. Development Methodology
4. Project Schedule
5. Implementation Phases
6. Milestones
7. Testing and Quality Assurance
8. Dependencies and Critical Path
9. Risk Management and Contingency
10. Final Deliverables and Submission Checklist
11. Conclusion

> This plan starts at Day 0 and describes planned activities only. Hours and task IDs are identical to the Project Estimation Document (114 hours in total).

1. Project Introduction and Objectives

The RAG Assistant is a web application that answers questions using only the documents a user uploads. It combines document parsing, vector search and a large language model (LLM), and shows citations so every answer can be checked against its source. The project is planned for one student over 38 working days.

Objectives:

1. Upload, parse and manage PDF, TXT and Markdown documents.
2. Chunk, embed and index document text in Qdrant.
3. Retrieve relevant chunks (dense search, with BM25 + RRF fusion and cross-encoder reranking as enhancements).
4. Generate grounded answers with an LLM and return citations.
5. Store conversation history and secure all data per user.
6. Deliver a tested, documented application and a demonstration.

2. Scope

2.1 Included features

Feature	Priority
User registration, login and JWT-protected API	Core
Upload PDF, TXT and Markdown files with validation	Core
Parsing, chunking, embeddings and Qdrant indexing	Core

Feature	Priority
Document list and delete (PostgreSQL metadata and Qdrant vectors)	Core
Question answering with an LLM, optionally limited to one document	Core
Citations and source snippets for every answer	Core
Conversation history (create, list, reopen, delete)	Core
Next.js web interface for all of the above	Core
Automated unit and API tests; README; UML diagrams	Core
Hybrid retrieval: BM25 + Reciprocal Rank Fusion	Enhancement (optional)
Cross-encoder reranking	Enhancement (optional)
Rewriting follow-up questions using history	Enhancement (optional)

2.2 Excluded or optional features

- **Excluded:** OCR for scanned PDFs; DOCX, PPTX and image files; streaming answers; sharing documents between users; admin dashboard; model fine-tuning; mobile app; production-scale hosting, monitoring and load testing.
- **Optional (only if required or time allows):** public deployment (about 4-6 h, not in the total); showing citations for reloaded conversations; retrieval evaluation metrics on a labelled question set.

3. Development Methodology

The project uses a simple **iterative, incremental** approach. Work is done phase by phase, and every phase adds a working slice of the system that is tested before the next phase builds on it.

1. **Plan:** pick the phase tasks from the WBS and confirm their dependencies are finished.
2. **Build:** implement the smallest working version first (for example dense retrieval before hybrid retrieval).
3. **Validate:** run unit tests or manual checks against the phase milestone criteria (Section 6).
4. **Review:** compare hours used with the estimate, note issues, and adjust the next phase or use contingency.
5. **Commit:** commit the working state to Git so there is always a version that runs.

A phase counts as complete only when its milestone criteria are met. Enhancements are built after the core path works end to end, so an unfinished enhancement never blocks a working demonstration.

4. Project Schedule

Work is planned sequentially at 3 hours per working day and 5 working days per week. Day 1 is the first working day after Day 0 (project start).

> **Schedule baseline check.** The suggested baseline of 14 working days at 2-3 hours per day provides only 28-42 hours. The bottom-up estimate is 114 hours, so the baseline cannot hold the full scope. The plan therefore uses 38 working days at 3 hours per day (about 8 weeks of 5 working days). Finishing in 14 working days would need about 9 hours per day, or a reduced scope (see the deprioritisation order).

4.1 Week-by-week plan

Week	Days	Phases	Tasks (hours)	Hours	Depends on (earlier work)	Deliverables / milestones
Week 1	Days 1-5	P1 Requirements and environment setup, P2 Architecture and database design, P3 Document ingestion and parsing	1.1 (1.5 h); 1.2 (1.5 h); 1.3 (1.5 h); 1.4 (1.5 h); 2.1 (2 h); 2.2 (2.5 h); 2.3 (1.5 h); 3.1 (3 h)	15	-	M1 Environment ready; M2 Design approved
Week 2	Days 6-10	P3 Document ingestion and parsing, P4 Chunking, embeddings and vector indexing	3.2 (2 h); 3.3 (1 h); 3.4 (2 h); 4.1 (2 h); 4.2 (2 h); 4.3 (3 h); 4.4 (2 h); 4.5 (1 h)	15	1.3, 1.4, 2.2, 2.3, 3.1	M3 Documents accepted; M4 Documents indexed
Week 3	Days 11-15	P5 Retrieval and answer generation, P6 Citation generation	5.1 (2.5 h); 5.2 (3.5 h); 5.3 (3 h); 5.4 (2 h); 5.5 (1 h); 6.1 (1.5 h); 6.2 (1.5 h)	15	4.4	M5 Questions answered; M6 Answers cited
Week 4	Days 16-20	P7 Authentication and conversation history, P8 Frontend integration	7.1 (3 h); 7.2 (2.5 h); 7.3 (2.5 h); 7.4 (1 h); 8.1 (2 h); 8.2 (4 h)	15	1.3, 2.2, 5.2	M7 Secure multi-user backend
Week 5	Days 21-25	P8 Frontend integration	8.3 (3.5 h); 8.4 (5 h); 8.5 (2 h); 8.6 (1.5 h); 8.7 (3 h)	15	3.4, 6.2, 7.3, 8.2	M8 Usable web application
Week 6	Days 26-30	P9 Testing and error handling	9.1 (4 h); 9.2 (4 h); 9.3 (3 h); 9.4 (4 h)	15	8.7, P3-P6, P7	M9 Quality verified
Week 7	Days 31-35	P10 Documentation, diagrams and final demonstration, C Contingency	10.1 (1.5 h); 10.2 (1.5 h); 10.3 (1.5 h); 10.4 (2 h); 10.5 (1.5 h); 10.6 (1 h); Contingency (6 h)	15	10.1-10.3, 8.7, 9.4	M10 Submission ready
Week 8	Days 36-38	C Contingency	Contingency (9 h)	9	-	Final submission

Total planned: 99 h of tasks + 15 h contingency = 114 h over 38 working days (38 x 3 h = 114 h of working capacity, which matches the estimate).

4.2 Gantt-style timeline (working days)

Phase	Hours	Week 1					Week 2					Week 3					Week 4					Week 5					Week 6					Week 7					Week 8		
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38
P1 Requirements and environment setup	6																																						
P2 Architecture and database design	6																																						
P3 Document ingestion and parsing	8																																						
P4 Chunking, embeddings and vector indexing	10																																						
P5 Retrieval and answer generation	12																																						
P6 Citation generation	3																																						
P7 Authentication and conversation history	9																																						
P8 Frontend integration	21																																						
P9 Testing and error handling	15																																						
P10 Documentation, diagrams and final demonstration	9																																						
C Contingency buffer	15																																						
Milestones (M#)			1		2			3			4				5	6			7							8					9			10					

Blue bars: planned phase work. Gold bar: contingency buffer (held at the end, can be used earlier if a phase overruns). Milestone numbers mark the day each phase is planned to finish. Phases run in sequence; a bar that shares a day with the next phase means the phase ends part-way through that day.

4.3 Day-by-day plan

Day	Week	Planned work (hours)	Milestone
Day 1	W1	1.1 Requirements analysis, scope and feature list (1.5 h); 1.2 Project estimation and project plan (this documentation) (1.5 h)	
Day 2	W1	1.3 Environment setup: Python, Node.js, Docker (PostgreSQL, Qdrant), API key (1.5 h); 1.4 Project skeleton: FastAPI app, settings/.env, logging, health endpoint, Git repo (1.5 h)	M1 Environment ready
Day 3	W1	2.1 System architecture, REST API design and data flow (2 h); 2.2 PostgreSQL schema (users, documents, conversations, messages) and initialisation (1 h)	
Day 4	W1	2.2 PostgreSQL schema (users, documents, conversations, messages) and initialisation (1.5 h); 2.3 Qdrant collection and payload design (user and document filters) (1.5 h)	M2 Design approved
Day 5	W1	3.1 Upload endpoint: file type, size and filename validation; file storage (3 h)	
Day 6	W2	3.2 PDF parser (page-wise text extraction with pypdf) (2 h); 3.3 TXT and Markdown parser (encoding handling) (1 h)	
Day 7	W2	3.4 Document metadata store; list, get and delete endpoints (2 h); 4.1 Text chunking with overlap and page tracking (1 h)	M3 Documents accepted
Day 8	W2	4.1 Text chunking with overlap and page tracking (1 h); 4.2 Embedding generation (sentence-transformers model) (2 h)	
Day 9	W2	4.3 Qdrant store: upsert, filtered similarity search, delete (3 h)	
Day 10	W2	4.4 Ingestion pipeline orchestration and cleanup on failure (2 h); 4.5 Verify indexing with sample documents (1 h)	M4 Documents indexed
Day 11	W3	5.1 Query endpoint with dense vector retrieval (2.5 h); 5.2 Prompt design and LLM integration (answers grounded in context) (0.5 h)	
Day 12	W3	5.2 Prompt design and LLM integration (answers grounded in context) (3 h)	
Day 13	W3	5.3 Hybrid retrieval: BM25 keyword search fused with Reciprocal Rank Fusion (3 h)	
Day 14	W3	5.4 Cross-encoder reranking of retrieved chunks (2 h); 5.5 Retrieval check on sample questions; tune chunk size and top-k (1 h)	M5 Questions answered
Day 15	W3	6.1 Context builder that labels chunks with source IDs [S1]..[Sn] (1.5 h); 6.2 Citation and source objects in the API response (1.5 h)	M6 Answers cited
Day 16	W4	7.1 User registration and login with password hashing (3 h)	
Day 17	W4	7.2 JWT authentication, route protection, per-user data isolation (2.5 h); 7.3 Conversation and message storage; history endpoints (0.5 h)	
Day 18	W4	7.3 Conversation and message storage; history endpoints (2 h); 7.4 Use recent history in queries; rewrite follow-up questions (1 h)	M7 Secure multi-user backend
Day 19	W4	8.1 Next.js, TypeScript and Tailwind CSS setup; application layout (2 h); 8.2 API client, login and registration pages, session handling (1 h)	
Day 20	W4	8.2 API client, login and registration pages, session handling (3 h)	

Day	Week	Planned work (hours)	Milestone
Day 21	W5	8.3 Document library: upload with progress, list, delete (3 h)	
Day 22	W5	8.3 Document library: upload with progress, list, delete (0.5 h); 8.4 Chat interface: messages, composer, loading/error states, document scope (2.5 h)	
Day 23	W5	8.4 Chat interface: messages, composer, loading/error states, document scope (2.5 h); 8.5 Citation markers and source panel (0.5 h)	
Day 24	W5	8.5 Citation markers and source panel (1.5 h); 8.6 Conversation history sidebar (1.5 h)	
Day 25	W5	8.7 Frontend-backend integration fixes and UI polish (3 h)	M8 Usable web application
Day 26	W6	9.1 Unit tests: parsers, chunking, retrieval fusion, reranking, citations (3 h)	
Day 27	W6	9.1 Unit tests: parsers, chunking, retrieval fusion, reranking, citations (1 h); 9.2 API tests: auth, upload, documents, query, history, user isolation (2 h)	
Day 28	W6	9.2 API tests: auth, upload, documents, query, history, user isolation (2 h); 9.3 End-to-end manual testing with sample documents (1 h)	
Day 29	W6	9.3 End-to-end manual testing with sample documents (2 h); 9.4 Error-handling hardening and bug fixing (1 h)	
Day 30	W6	9.4 Error-handling hardening and bug fixing (3 h)	M9 Quality verified
Day 31	W7	10.1 README and setup instructions (1.5 h); 10.2 UML class diagram and activity diagram (1.5 h)	
Day 32	W7	10.3 Test evidence and screenshots (1.5 h); 10.4 Final report; update estimation and plan with actual progress (1.5 h)	
Day 33	W7	10.4 Final report; update estimation and plan with actual progress (0.5 h); 10.5 Demonstration preparation and rehearsal (1.5 h); 10.6 Final review and submission (1 h)	M10 Submission ready
Day 34	W7	Contingency buffer (3 h)	
Day 35	W7	Contingency buffer (3 h)	
Day 36	W8	Contingency buffer (3 h)	
Day 37	W8	Contingency buffer (3 h)	
Day 38	W8	Contingency buffer (3 h)	

5. Implementation Phases

Phase	Main work	Tasks	Hours	Days	Phase output (validated)
P1 Requirements and environment setup	Clarify requirements, write the estimation and plan, install tools and create the project skeleton.	1.1, 1.2, 1.3, 1.4	6	1-2	Health endpoint responds; databases run in Docker; plan approved.
P2 Architecture and database design	Design the architecture, API endpoints, PostgreSQL tables and the Qdrant collection and payload.	2.1, 2.2, 2.3	6	3-4	Schema initialises without errors; API list and data flow documented.
P3 Document ingestion and parsing	Build the upload endpoint with validation and the PDF and text parsers; store document metadata.	3.1, 3.2, 3.3, 3.4	8	5-7	Sample files of each type parse into page text; invalid uploads are rejected with clear errors.
P4 Chunking, embeddings and vector indexing	Chunk text with overlap, generate embeddings and store them in Qdrant; orchestrate the ingestion pipeline with cleanup on failure.	4.1, 4.2, 4.3, 4.4, 4.5	10	7-10	Sample documents are searchable in Qdrant; chunk and page counts are correct.
P5 Retrieval and answer generation	Build the query endpoint: dense retrieval, prompt design and LLM integration; add BM25 + RRF and reranking as enhancements.	5.1, 5.2, 5.3, 5.4, 5.5	12	11-14	Sample questions return relevant chunks and grounded answers.
P6 Citation generation	Label retrieved chunks with source IDs and return structured citations and sources.	6.1, 6.2	3	15	Citation IDs in answers match the returned sources.
P7 Authentication and conversation history	Add registration, login, JWT protection, per-user isolation and conversation history.	7.1, 7.2, 7.3, 7.4	9	16-18	Unauthenticated and cross-user requests are rejected; history persists.
P8 Frontend integration	Build the Next.js interface: auth pages, document library, chat, citations and history sidebar.	8.1, 8.2, 8.3, 8.4, 8.5, 8.6, 8.7	21	19-25	All main flows work from the browser against the real backend.
P9 Testing and error handling	Write unit and API tests, run end-to-end checks and fix defects.	9.1, 9.2, 9.3, 9.4	15	26-30	Tests pass; acceptance criteria met or deviations documented.
P10 Documentation, diagrams and final demonstration	Write the README, draw UML diagrams, collect test evidence, prepare the report and demo.	10.1, 10.2, 10.3, 10.4, 10.5, 10.6	9	31-33	Submission checklist complete.

Detailed task descriptions, hours and dependencies are listed in the Work Breakdown Structure of the Project Estimation Document.

6. Milestones

ID	Milestone	Planned day	Completion criteria (measurable)
M1	Environment ready	Day 2	Backend health endpoint responds; PostgreSQL and Qdrant containers run; estimation and plan drafted.
M2	Design approved	Day 4	Schema creates all four tables; API endpoint list and Qdrant payload documented.
M3	Documents accepted	Day 7	PDF, TXT and MD files upload and parse into page text; invalid type, empty and oversized files are rejected.
M4	Documents indexed	Day 10	Uploaded sample documents produce chunks with page numbers and vectors stored in Qdrant; chunk count recorded.
M5	Questions answered	Day 14	A question about a sample document returns an LLM answer built from retrieved chunks.
M6	Answers cited	Day 15	Each answer carries citations with document, page and chunk that match the retrieved context.
M7	Secure multi-user backend	Day 18	Protected endpoints reject missing or invalid tokens; one user cannot read another user's documents or chats; history is stored.
M8	Usable web application	Day 25	A user can sign in, upload, ask, view citations and reopen past conversations from the browser.
M9	Quality verified	Day 30	Automated unit and API tests pass; end-to-end checklist completed; known defects fixed or documented.
M10	Submission ready	Day 33	README, diagrams, estimation, plan, test evidence and demo are complete and checked against the submission checklist.
-	Final submission	Day 38	All deliverables submitted; contingency used or remaining time noted.

7. Testing and Quality Assurance

7.1 Test levels

Level	What is tested	Tools / method	When
Unit testing	Parsers, chunking, embedding wrapper, BM25/RRF fusion, reranking order, context and citation building, password hashing and tokens	pytest with small fixtures; LLM and models replaced by fakes where needed	During each phase (P3-P7) and task 9.1
API testing	Auth, upload, documents, query, history endpoints: success cases, validation errors, 401/403/404, user isolation	pytest + FastAPI TestClient (httpx)	Phases P7-P9, task 9.2
Integration testing	Upload to parse to chunk to embed to Qdrant; query to retrieval to LLM to citations, with real PostgreSQL and Qdrant	Scripted runs against Docker services with sample documents	End of P4, P5, P6; task 9.3
End-to-end validation	Full user journeys in the browser	Manual test checklist with screenshots	Task 9.3 and before the demo
Static checks	Type errors and lint in the frontend	tsc --noEmit, ESLint	Phase P8

7.2 Acceptance criteria

Area	Acceptance criteria
Document upload	A signed-in user uploads a valid PDF, TXT or MD file and sees it in the document list with page and chunk counts. Unsupported types, empty files and files over the size limit are rejected with a clear message and nothing is stored.
Question answering	A question about an uploaded document returns an answer based on that document's content. If nothing relevant is found, the system says so instead of guessing. Questions can be limited to one document.
Citations	Each answer lists its sources (document, page, chunk). Every citation marker in the answer refers to a returned source, and the source text supports the cited statement when checked manually.
Conversation history	Questions and answers are saved to a conversation; the user can reopen it later; follow-up questions use earlier context; deleting a conversation removes its messages.
Authentication and isolation	Endpoints reject requests without a valid token; a user cannot see, query or delete another user's documents or conversations.
Error handling	Invalid input returns 4xx responses with readable messages; service failures (LLM, database, vector store) return an error without crashing the server; a failed upload leaves no partial data behind.

> No test results are reported in this document. Test outcomes will be recorded during Phase 9 and included as test evidence (task 10.3).

8. Dependencies and Critical Path

8.1 Key dependencies

- Environment setup (1.3) and the project skeleton (1.4) come before all development.
- The PostgreSQL schema (2.2) is needed by upload (3.1), document metadata (3.4), authentication (7.1) and history (7.3).
- Parsers (3.2, 3.3) feed chunking (4.1); chunking, embeddings and the Qdrant store (4.2, 4.3) feed the ingestion pipeline (4.4).
- Retrieval and answering (5.1, 5.2) need indexed documents (4.4); citations (6.1, 6.2) need answers (5.2).
- Frontend pages need the matching backend endpoints: auth (7.2) for 8.2, documents (3.4) for 8.3, citations (6.2) for 8.4-8.5, history (7.3) for 8.6.
- Final testing (9.3) needs the integrated frontend (8.7); documentation (P10) needs testing to be finished (9.4).

8.2 Critical path

1.1 → 1.3 → 1.4 → 2.2 → 3.1 → 4.4 → 5.1 → 5.2 → 6.1 → 6.2 → 8.4 → 8.5 → 8.7 → 9.3 → 9.4 → 10.2 → 10.4 → 10.5 → 10.6

Because one student does all the work, tasks run one after another and any delay on this chain delays submission. Tasks that are not on the critical path (for example 5.3 hybrid retrieval, 5.4 reranking and 7.4 query rewriting) can be moved or dropped without delaying the end date.

8.3 Main schedule bottlenecks

- **Ingestion pipeline (4.4):** combines parsing, chunking, embeddings and Qdrant; first model download and Docker setup can cause delays.
- **LLM integration (5.2):** depends on an external API, key setup and prompt tuning.
- **Chat interface and integration (8.4, 8.7):** the largest frontend tasks, where API contract mismatches appear.
- **Bug fixing (9.4):** hard to predict; protected by the contingency buffer.

9. Risk Management and Contingency

The main risks, their likelihood, impact and mitigation are listed in the Risk Assessment of the Project Estimation Document (R1-R11). Risks are reviewed at the end of each week against the milestone plan.

9.1 Contingency

- 15 hours (5 working days) are reserved at the end of the schedule (Days 34-38).
- If a phase overruns by more than half a day, the overrun is taken from contingency and recorded in the weekly review.
- If more than half of the contingency is used before Week 6, the deprioritisation order below is applied.

9.2 Deprioritisation order (if the deadline becomes tight)

1. Optional public deployment (not scheduled).
2. 7.4 Follow-up question rewriting (1 h): keep plain history only.
3. 5.4 Cross-encoder reranking (2 h): use the fused ranking directly.
4. 5.3 BM25 + RRF hybrid retrieval (3 h): use dense vector search only.
5. UI polish within 8.7: keep functional screens, reduce styling work.
6. Reduce frontend unit tests; keep backend unit and API tests for the core path.

The core path (upload, indexing, retrieval, LLM answer, citations, authentication, history, tests and documentation) is never cut, because it is what the assignment is assessed on. Dropping items 2-4 saves 6 hours.

10. Final Deliverables and Submission Checklist

☐ Working application (backend, frontend, PostgreSQL and Qdrant run with the documented commands)

☐ Source code in a Git repository (backend, frontend, tests)

☐ README with setup, configuration (.env) and run instructions

☐ UML class diagram

☐ UML activity diagram

☐ Project Estimation Document

☐ Project Plan (this document)

☐ Test evidence: automated test output and end-to-end checklist results

☐ Screenshots of key flows: login, upload, question with citations, history

☐ Demonstration (live or recorded); deployment link only if required

☐ Final review against acceptance criteria and submission

11. Conclusion

The plan builds the RAG Assistant in ten iterative phases over 38 working days (114 hours including 15 hours of contingency). Core features (document ingestion, vector search, grounded answers with citations, authentication and history) are scheduled first, and retrieval enhancements (hybrid search, reranking, query rewriting) come after the core path works. Each phase ends with measurable milestone criteria and tests, so progress can be checked objectively. If the plan is followed, the expected outcome is a working, tested and documented application ready for demonstration; actual results will be recorded during development and are not assumed here.